

**LAPORAN PRATIKUM STRUKTUR DATA
IMPLEMENTASI SHORTING MENGGUNAKAN SHELL
SORT, QUICK SORT, DAN MERGE SORT**

disusun Oleh:

AFIF RAHMAN SALEH

NIM 251153105

Dosen Pengampu: Dr. WAHYUDI, S.T, M.T

Asisten Praktikum : M Galid Alvero



**DEPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS**

TAHUN

2026

KATA PENGANTAR

Puji syukur ke hadirat Allah SWT atas rahmat-Nya sehingga laporan praktikum Struktur Data ini dapat diselesaikan. Laporan ini disusun sebagai bentuk pertanggungjawaban kegiatan praktikum pekan pertama dengan topik implementasi shorting menggunakan shell sort, quick sort, dan merge sort.

Ucapan terima kasih penulis sampaikan kepada dosen pengampu, asisten praktikum, serta rekan-rekan mahasiswa atas bimbingan dan kerja sama yang diberikan. Penulis menyadari laporan ini masih jauh dari sempurna, sehingga kritik dan saran sangat diharapkan demi perbaikan di masa mendatang.

Semoga laporan ini bermanfaat bagi penulis maupun pembaca dalam memahami konsep struktur data dan penerapannya dalam pemrograman.

DAFTAR ISI

KATA PENGANTAR.....	i
DAFTAR ISI	ii
BAB I PENDAHULUAN	1
1.1 Latar Belakang	1
1.2 Tujuan Praktikum	2
1.3 Manfaat Praktikum.....	2
BAB II PEMBAHASAN	3
2.1 Landasan Teori	3
2.2 Kode Java yang Dipraktikkan	5
2.3 Penjelasan Langkah Kerja.....	12
2.4 Analisis Hasil	16
BAB III KESIMPULAN.....	19
5.1 Kesimpulan Praktikum.....	19
DAFTAR PUSTAKA	20

BAB I

PENDAHULUAN

1.1 Latar Belakang

Dalam pemrograman, pengolahan data merupakan salah satu aspek penting untuk menghasilkan program yang efisien dan terstruktur. Data yang tidak terurut sering kali menyulitkan proses pencarian, analisis, maupun pengolahan lebih lanjut. Oleh karena itu, diperlukan suatu metode untuk mengurutkan data agar lebih mudah dikelola. Proses pengurutan data tersebut dikenal dengan istilah *sorting algorithm* atau algoritma pengurutan.

Pada praktikum ini kita mempraktikkan tiga algoritma sorting, yaitu Shell Sort, Quick Sort, dan Merge Sort. Ketiga algoritma tersebut termasuk algoritma sorting yang lebih efisien dibandingkan metode sorting sederhana seperti Bubble Sort, Selection Sort, dan Insertion Sort karena mampu menangani data dalam jumlah yang lebih besar dengan waktu proses yang lebih cepat.

Shell Sort merupakan pengembangan dari Insertion Sort yang bekerja dengan membandingkan elemen-elemen berdasarkan jarak tertentu (gap) sehingga proses pengurutan menjadi lebih cepat. Quick Sort menggunakan metode divide and conquer dengan menentukan sebuah pivot untuk membagi data menjadi beberapa bagian sebelum dilakukan pengurutan secara rekursif. Sedangkan Merge Sort juga menerapkan metode divide and conquer dengan cara membagi array menjadi beberapa bagian kecil, kemudian menggabungkannya kembali dalam keadaan terurut.

Selain implementasi algoritma sorting berbasis console, pada praktikum ini juga dilakukan pembuatan program berbasis Graphical User Interface (GUI) menggunakan Java Swing untuk memvisualisasikan proses Bubble Sort secara bertahap (step by step). Visualisasi tersebut bertujuan agar proses pengurutan data dapat dipahami dengan lebih mudah dan interaktif.

Melalui praktikum ini, mahasiswa diharapkan dapat memahami konsep dasar algoritma sorting, mengetahui perbedaan metode kerja masing-masing algoritma, serta mampu mengimplementasikannya ke dalam bahasa pemrograman Java.

1.2 Tujuan Praktikum

1. Memahami konsep dasar algoritma pengurutan data (*sorting*) dalam pemrograman Java.
2. Mengetahui cara kerja algoritma Shell Sort, Quick Sort, dan Merge Sort dalam mengurutkan data.
3. Mengimplementasikan algoritma sorting ke dalam program Java berbasis console.
4. Memahami penggunaan rekursi pada algoritma Quick Sort dan Merge Sort.
5. Membuat visualisasi proses sorting menggunakan GUI Java Swing pada algoritma Bubble Sort.

1.3 Manfaat Praktikum

1. Menambah pemahaman mengenai konsep algoritma pengurutan data (*sorting*).
2. Melatih kemampuan dalam mengimplementasikan algoritma sorting menggunakan bahasa pemrograman Java.
3. Membantu memahami perbedaan metode kerja antara Shell Sort, Quick Sort, Merge Sort, dan Bubble Sort.
4. Meningkatkan kemampuan logika pemrograman melalui penggunaan metode rekursif pada proses sorting.
5. Memberikan pengalaman dalam membuat aplikasi berbasis GUI Java Swing untuk visualisasi algoritma sorting.mendatang.

BAB II

PEMBAHASAN

2.1 Landasan Teori

Sorting adalah proses mengurutkan data ke dalam urutan tertentu, baik secara menaik (*ascending*) maupun menurun (*descending*). Dalam struktur data dan algoritma, sorting sangat penting karena dapat mempermudah proses pencarian data dan meningkatkan efisiensi pengolahan data. Pada praktikum ini terdapat beberapa algoritma sorting dasar, yaitu Shell Sort, Quick Sort, Merge Sort, dan visualisasi Bubble Sort berbasis GUI.

Yang pertama adalah Shell Sort, Shell Sort merupakan pengembangan dari algoritma Insertion Sort yang menggunakan konsep pengelompokan data berdasarkan jarak tertentu (*gap*). Algoritma ini bekerja dengan membandingkan elemen-elemen yang memiliki jarak tertentu terlebih dahulu, kemudian secara bertahap memperkecil nilai *gap* hingga menjadi 1.

Kelebihan dari Shell Sort adalah mampu mempercepat proses pengurutan dibandingkan Insertion Sort karena perpindahan data dapat dilakukan lebih cepat melalui pengelompokan jarak.

Langkah kerja Shell Sort:

1. Menentukan nilai *gap* awal, biasanya setengah dari jumlah data.
2. Membandingkan elemen berdasarkan nilai *gap*.
3. Melakukan pertukaran apabila data tidak sesuai urutan.
4. Mengurangi nilai *gap* hingga bernilai 1.
5. Menghasilkan data yang telah terurut.

Yang kedua yaitu Quick Sort, Quick Sort merupakan algoritma sorting yang menggunakan metode Divide and Conquer. Algoritma ini bekerja dengan memilih satu elemen sebagai pivot, kemudian membagi data menjadi dua bagian, yaitu data yang lebih kecil dari pivot dan data yang lebih besar dari pivot.

Quick Sort termasuk algoritma sorting yang cepat karena proses pembagian data dilakukan secara rekursif sehingga kompleksitas rata-rata algoritma ini cukup efisien.

Langkah kerja Quick Sort:

1. Memilih sebuah elemen sebagai pivot.
2. Membagi array menjadi dua bagian berdasarkan pivot.
3. Mengurutkan bagian kiri dan kanan secara rekursif.
4. Menggabungkan hasil hingga data terurut seluruhnya.

Yang ketiga yaitu Merge Sort, Merge Sort merupakan algoritma sorting yang juga menggunakan metode Divide and Conquer. Algoritma ini bekerja dengan membagi data menjadi bagian-bagian kecil sampai setiap bagian hanya memiliki satu elemen, kemudian bagian tersebut digabungkan kembali dalam keadaan terurut.

Keunggulan Merge Sort terletak pada kestabilannya dalam mengurutkan data dan efisiensinya untuk data berukuran besar.

Langkah kerja Merge Sort:

1. Membagi array menjadi dua bagian.
2. Membagi kembali hingga tersisa satu elemen.
3. Menggabungkan elemen dengan urutan yang benar.
4. Menghasilkan array yang telah terurut.

Dan yang terakhir adalah Bubble Sort merupakan algoritma sorting sederhana yang bekerja dengan membandingkan dua elemen yang berdekatan dan menukarnya apabila urutan data salah. Pada praktikum ini, Bubble Sort diterapkan dalam bentuk GUI Java Swing untuk memvisualisasikan proses pengurutan data secara bertahap (*step by step*).

Visualisasi GUI bertujuan untuk membantu pengguna memahami proses pertukaran data pada algoritma Bubble Sort secara lebih interaktif.

2.2 Kode Java yang Dipraktikkan

1. Kelas shellsort_2511531005

```
package pekan8_2511531005;

public class shellsort_2511531005 {

    public static void shellsort_2511531005(int[] A_1005) {

        int n_1005 = A_1005.length;
        int gap_1005 = n_1005/2;
        while (gap_1005 > 0) {
            for(int i = gap_1005; i < n_1005; i++) {
                int temp_1005 = A_1005[i];
                int j = i;
                while (j >= gap_1005 && A_1005[j - gap_1005] >
temp_1005) {
                    A_1005[j] = A_1005[j - gap_1005];
                    j = j - gap_1005;
                }
                A_1005[j] = temp_1005;
            }
            gap_1005= gap_1005/2;
        }
    }

    public static void main(String[] args) {
        // TODO Auto-generated method stub

        int[] data = {3, 10, 4, 6, 8, 9, 7, 2, 1, 5};
        System.out.println("sebelum : ");
        printArray(data);

        shellsort_2511531005(data);

        System.out.println("sesudah shellsort : ");
        printArray(data);
    }

    public static void printArray(int[] arr) {
        for (int i: arr) System.out.print(i + " ");
        System.out.println();
    }
}
```

1. Kelas quicksort_2511531005

```
package pekan8_2511531005;
```



```

• public class quicksort_2511531005 {
•
•     static void swap_1005 (int[] arr_1005, int i_1005, int j_1005) {
•         int temp_1005 = arr_1005[i_1005];
•         arr_1005[i_1005] = arr_1005[j_1005];
•         arr_1005[j_1005] = temp_1005;
•     }
•
•     static void medianOfThree_1005(int [] arr_1005, int low_1005, int
high_1005) {
•         int mid_1005 = low_1005 + (high_1005 - low_1005) / 2;
•
•         if (arr_1005[low_1005] > arr_1005[mid_1005]) {
•             swap_1005(arr_1005, low_1005, mid_1005);
•         }
•         if (arr_1005[low_1005] > arr_1005[high_1005]) {
•             swap_1005(arr_1005, low_1005, high_1005);
•         }
•         if (arr_1005[mid_1005] > arr_1005[high_1005]) {
•             swap_1005(arr_1005, mid_1005, high_1005);
•         }
•         swap_1005(arr_1005, mid_1005, high_1005);
•     }
•
•     static int partition_1005(int[] arr_1005, int low_1006, int high_1005) {
•         medianOfThree_1005(arr_1005, low_1006, high_1005);
•
•         int pivot = arr_1005[high_1005];
•         int i = (low_1006 - 1);
•
•         for (int j = low_1006; j<= high_1005-1; j++) {
•             if (arr_1005[j] < pivot) {
•                 i++;
•                 swap_1005(arr_1005, i, j);
•             }
•         }
•
•         swap_1005(arr_1005, i + 1, high_1005);
•         return(i +1);
•     }
•
•     static void quicksort_1005 (int[] arr_1005, int low_1005, int high_1005)
{
•         if (low_1005 < high_1005) {
•             int pi = partition_1005(arr_1005, low_1005, high_1005);
•             quicksort_1005(arr_1005, low_1005, pi -1 );
•             quicksort_1005(arr_1005, pi + 1, high_1005);
•         }
•     }
•
•     public static void printArr_1005(int[] arr_1005) {
•         for(int i = 0; i< arr_1005.length;i++){
•             System.out.print(arr_1005[i] + " ");
•         }
•         System.out.println();
•     }
• }

```

```

•
•
•   public static void main(String[] args) {
•       // TODO Auto-generated method stub
•       int[] arr_1005= {10,7,8,9,1,5};
•       int N_1005 = arr_1005.length;
•       System.out.println("data sebelum diurutkan :");
•       printArr_1005(arr_1005);
•
•       quicksort_1005(arr_1005, 0 , N_1005-1);
•
•       System.out.println("data terurut quicksort :");
•       printArr_1005(arr_1005);
•   }
•
• }

```

1. Kelas margesort_2511531005

```

•
•   package pekan8_2511531005;
•
•   public class margesort_2511531005 {
•
•       void marge_1005(int arr[], int l, int m, int r) {
•           int n1_1005 = m - l + 1;
•           int n2_1005 = r - m;
•
•           int L[] = new int[n1_1005];
•           int R[] = new int[n2_1005];
•
•           for(int i = 0; i < n1_1005; ++i)
•               L[i] = arr[l + i];
•           for(int j = 0; j < n2_1005; ++j)
•               R[j] = arr[m + 1 + j];
•
•           int i_1005 = 0, j = 0;
•           int k_1005 = l;
•
•           while (i_1005 < n1_1005 && j < n2_1005) {
•               if (L[i_1005] <= R[j]) {
•                   arr[k_1005] = L[i_1005];
•                   i_1005++;
•               }else {
•                   arr[k_1005] = R[j];
•                   j++;
•               }
•               k_1005++;
•           }
•
•           while(i_1005 < n1_1005) {
•               arr[k_1005] = L[i_1005];
•               i_1005++;
•               k_1005++;
•           }
•       }
•   }

```

```

    while(j < n2_1005) {
        arr[k_1005] = R[j];
        j++;
        k_1005++;
    }
}

void sort_1005(int arr[], int l, int r) {
    if(l < r) {
        int m_1005 = (l + r) / 2;

        sort_1005(arr, l, m_1005);
        sort_1005(arr, m_1005 + 1, r);

        marge_1005(arr, l, m_1005, r);
    }
}

static void printArr_1005(int arr[]) {
    int n_1005 = arr.length;
    for(int i = 0; i < n_1005; ++i)
        System.out.print(arr[i] + " ");
    System.out.println();
}

public static void main(String[] args) {
    int arr[] = {12,11,13,5,6,7};

    System.out.println("sebelum terurut :");
    printArr_1005(arr);

    margsort_2511531005 ob = new margsort_2511531005();
    ob.sort_1005(arr, 0 , arr.length-1);

    System.out.println("sesudah terurut menggunakan margsort :");
    printArr_1005(arr);
}
}

```

1. Kelas bubblesortGUI__2511531005

```

1 package pekan8_2511531005;
2
3 import java.awt.BorderLayout;
4
5 public class bubblesortGUI__2511531005 extends JFrame {
6
7     private static final long serialVersionUID = 1L;
8     private JPanel contentPane;
9     private int[] array_1005;
10    private JLabel[] labelArray_1005;
11    private JButton stepButton_1005, resetButton_1005, setButton_1005;
12    private JTextField inputField_1005;
13    private JPanel panelArray_1005;
14    private JTextArea stepArea_1005;
15
16    private int i = 1, j;
17    private boolean sorting_1005 = false;
18    private int stepCount_1005 = 1;
19
20 }

```

```

42 public bubblesortGUI_2511531005() {
43     setTitle("insertion sort langkah per langkah");
44     setSize(750, 400);
45     setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
46     setLocationRelativeTo(null);
47     setLayout(new BorderLayout());
48
49     JPanel inputPanel_1005 = new JPanel(new FlowLayout());
50     inputField_1005 = new JTextField(30);
51     setButton_1005 = new JButton("set array");
52     inputPanel_1005.add(new JLabel("masukkan angka (pisahkan dengan koma) : "));
53     inputPanel_1005.add(inputField_1005);
54     inputPanel_1005.add(setButton_1005);
55
56     panelArray_1005 = new JPanel();
57     panelArray_1005.setLayout(new FlowLayout());
58
59     JPanel controlPanel = new JPanel();
60     stepButton_1005 = new JButton("langkah selanjutnya ");
61     resetButton_1005 = new JButton("reset");
62     stepButton_1005.setEnabled(false);
63     controlPanel.add(stepButton_1005);
64     controlPanel.add(resetButton_1005);
65
66     stepArea_1005 = new JTextArea(8, 40);
67     stepArea_1005.setEditable(false);
68     stepArea_1005.setFont(new Font ("Monospaced", Font.PLAIN, 14));
69     JScrollPane scrollPane = new JScrollPane(stepArea_1005);
70
71     add(inputPanel_1005, BorderLayout.NORTH);
72     add(panelArray_1005, BorderLayout.CENTER);
73     add(controlPanel, BorderLayout.SOUTH);
74     add(scrollPane, BorderLayout.EAST);
75
76     setButton_1005.addActionListener(e -> setArrayFromInput_1005());
77
78     stepButton_1005.addActionListener(e -> performStep_1005());
79
80     resetButton_1005.addActionListener(e -> reset_1005());
81
82 }
83

```

```

84 private void setArrayFromInput_1005() {
85
86     String text = inputField_1005.getText().trim();
87     if(text.isEmpty()) return;
88     String[] parts = text.split(",");
89     array_1005 = new int[parts.length];
90     try {
91         for (int k = 0; k < parts.length; k++) {
92             array_1005[k] = Integer.parseInt(parts[k].trim());
93         }
94     }
95     catch (NumberFormatException e) {
96         JOptionPane.showMessageDialog(this, "masukkan hanya angka"
97             + " yang dipisahkan koma! ", "error", JOptionPane.ERROR_MESSAGE);
98         return;
99     }
100
101     i = 0;
102     j = 0;
103     stepCount_1005 = 1;
104     sorting_1005 = true;
105     stepButton_1005.setEnabled(true);
106     stepArea_1005.setText("");
107     panelArray_1005.removeAll();
108     labelArray_1005 = new JLabel[array_1005.length];
109
110     for (int k = 0; k < array_1005.length; k++) {
111         labelArray_1005[k] = new JLabel(String.valueOf(array_1005[k]));
112         labelArray_1005[k].setFont(new Font ("Arial", Font.BOLD, 24));
113         labelArray_1005[k].setOpaque(true);
114         labelArray_1005[k].setBackground(Color.white);
115         labelArray_1005[k].setBorder(BorderFactory.createLineBorder(Color.BLACK));
116         labelArray_1005[k].setPreferredSize(new Dimension(50, 50));
117         labelArray_1005[k].setHorizontalAlignment(SwingConstants.CENTER);
118         panelArray_1005.add(labelArray_1005[k]);
119     }
120
121     panelArray_1005.revalidate();
122     panelArray_1005.repaint();
123 }
124

```

```

125 public void performStep_1005() {
126     if (!sorting_1005 || i >= array_1005.length - 1) {
127         sorting_1005 = false;
128         stepButton_1005.setEnabled(false);
129         JOptionPane.showMessageDialog(this, "sorting selesai!");
130         return;
131     }
132     resetHighlights_1005();
133     StringBuilder stepLog = new StringBuilder();
134     labelArray_1005[j].setBackground(Color.CYAN);
135     labelArray_1005[j+1].setBackground(Color.CYAN);
136
137     if(array_1005[j] > array_1005[j+1]) {
138         int temp_1005 = array_1005[j];
139         array_1005[j] = array_1005[j + 1];
140         array_1005[j + 1] = temp_1005;
141         labelArray_1005[j].setBackground(Color.RED);
142         labelArray_1005[j + 1].setBackground(Color.RED);
143         stepLog.append("langkah ").append(stepCount_1005).append(": menukar elemen ke-")
144             .append(j).append(" ").append(array_1005[j+1]).append(" dengan elemen ke-")
145             .append(j+1).append(" ").append(array_1005[j]).append("\n");
146     }
147     else{
148         stepLog.append("langkah ").append(stepCount_1005).append(": tidak ada pertukaran ke-")
149             .append(j).append(" dan ke-").append(j+1).append("\n");
150
151         stepLog.append("hasil : ").append(arrayToString_1005(array_1005)).append("\n\n");
152         stepArea_1005.append(stepLog.toString());
153         updatelabels_1005();
154         j++;
155         if(j >= array_1005.length - i - 1 ) {
156             j = 0;
157             i++;
158             stepCount_1005++;
159             if(i >= array_1005.length - 1) {
160                 sorting_1005 = false;
161                 stepButton_1005.setEnabled(false);
162                 JOptionPane.showMessageDialog(this, "sorting selesai");
163             }
164         }
165     }
166 }
167
168 private void updatelabels_1005() {
169     for (int k = 0; k < array_1005.length; k++) {
170         labelArray_1005[k].setText(String.valueOf(array_1005[k]));
171     }
172 }
173
174
175 public void resetHighlights_1005() {
176     for(JLabel label : labelArray_1005) {
177         label.setBackground(Color.WHITE);
178     }
179 }
180
181 private void reset_1005() {
182     inputField_1005.setText("");
183     panelArray_1005.removeAll();
184     panelArray_1005.revalidate();
185     panelArray_1005.repaint();
186     stepArea_1005.setText("");
187     stepButton_1005.setEnabled(false);
188     sorting_1005 = false;
189     i = 0;
190     j = 0;
191     stepCount_1005 = 1;
192 }
193
194 private Object arrayToString_1005(int[] arr_1005) {
195     StringBuilder sb_1005 = new StringBuilder();
196     for(int k = 0; k < arr_1005.length; k++) {
197         sb_1005.append(arr_1005[k]);
198         if (k < arr_1005.length - 1) sb_1005.append(", ");
199     }
200     return sb_1005.toString();
201 }
202
203 public static void main (String[] args) {
204     SwingUtilities.invokeLater(() -> {
205         bubblesortGUI_2511531005 gui = new bubblesortGUI_2511531005();
206         gui.setVisible(true);
207     });
208 }
209
210
211
212
213 }

```

2.3 Penjelasan Langkah Kerja

1. Kelas shellsort_2511531005

- Method `public static void shellsort_2511531005(int[] A_1005)`
 - Method ini digunakan untuk melakukan proses pengurutan menggunakan algoritma Shell Sort.

Langkah langkah program :

- a. Menentukan panjang array menggunakan :
`int n_1005 = A_1005.length;`
 - b. Menentukan nilai awal gap, yaitu setengah dari jumlah data:
`int gap_1005 = n_1005/2;`
 - c. Membandingkan elemen dengan jarak tertentu (*gap*).
 - d. Melakukan perpindahan elemen apabila data sebelumnya lebih besar.
 - e. Mengurangi nilai gap hingga bernilai 1.
 - f. Menghasilkan array yang telah terurut.
- Method `public static void printArray(int[] arr)`
 - Method ini digunakan untuk menampilkan isi array sebelum dan sesudah proses sorting.
 - Main method
Method ini digunakan untuk :
 - Mendeklarasikan array data
 - Menampilkan data sebelum sorting
 - Menjalankan Shell Sort
 - Menampilkan hasil pengurutan

2. Kelas quicksort_2511531005

- Method `static void swap_1005`
 - Method ini digunakan untuk menukar posisi dua elemen array.

Langkah kerja:

- a. Menyimpan nilai sementara ke variable `int temp_1005`.
- b. Menukar posisi elemen pertama dan kedua.
- c. Menyimpan hasil pertukaran.
- Method `static void medianOfThree_1005`
 - Method ini digunakan untuk menentukan pivot menggunakan metode *median of three*.
 - Fungsi method :
 - Mengambil nilai tengah array.
 - Membandingkan elemen awal, tengah, dan akhir.
 - Menentukan pivot yang lebih optimal.
- Method partition `static int partition_1005`
 - Method ini digunakan untuk membagi array berdasarkan nilai pivot.

Langkah kerja :

- a. Menentukan pivot.
- b. Membandingkan seluruh elemen dengan pivot.
- c. Memindahkan elemen lebih kecil ke kiri.
- d. Menempatkan pivot pada posisi benar.
- Method `static void quicksort_1005`
 - Method ini menjalankan proses Quick Sort secara rekursif.

Langkah kerja:

- d. Membagi array menjadi dua bagian.
- e. Mengurutkan sisi kiri pivot.
- f. Mengurutkan sisi kanan pivot.
- g. Menghasilkan data yang terurut.
- Method `public static void printArr 1005`
 - Digunakan untuk menampilkan isi array ke layar.
- Main method
 - Method utama digunakan untuk:

- Menampilkan data sebelum sorting
- Menjalankan Quick Sort
- Menampilkan hasil sorting

3. Kelas mergesort_2511531005

- Method `void marge_1005`
 - Method ini digunakan untuk menggabungkan dua bagian array yang telah diurutkan.

Langkah kerja:

- a. Membuat array sementara kiri dan kanan.
 - b. Menyalin data ke array sementara.
 - c. Membandingkan elemen kiri dan kanan.
 - d. Menempatkan elemen terkecil ke array utama.
 - e. Menggabungkan seluruh data menjadi terurut.
- Method `void sort_1005`
 - Method ini digunakan untuk membagi array menjadi bagian kecil secara rekursif.

Langkah kerja:

- a. Membagi array menjadi dua bagian.
 - b. Melakukan pembagian ulang hingga tersisa satu elemen.
 - c. Menggabungkan data menggunakan method `marge_1005()`.
- Method `static void printArr_1005`
 - Method ini digunakan untuk menampilkan isi array.
 - Main method
 - Method utama digunakan untuk:
 - Menampilkan data sebelum sorting
 - Menjalankan Merge Sort
 - Menampilkan hasil sorting

4. Kelas bubblesortGUI__2511531005

- Konstruktor `public bubblesortGUI__2511531005()`
 - Constructor digunakan untuk membangun tampilan GUI yang terdiri dari:

- Input array angka
- Tombol set array
- Tombol langkah selanjutnya
- Tombol reset
- Area visualisasi array
- Area log langkah sorting
- Method `private void setArrayFromInput_1005()`
 - Method ini digunakan untuk:
 - Mengambil input angka dari pengguna.
 - Memisahkan angka berdasarkan tanda koma.
 - Mengubah data menjadi array integer.
 - Menampilkan visualisasi array pada panel GUI.
- Method `public void performStep_1005()`
 - Method ini digunakan untuk menjalankan Bubble Sort langkah demi langkah.

Langkah kerja:

- a. Membandingkan dua elemen berdekatan.
 - b. Menukar elemen jika urutannya salah.
 - c. Menampilkan proses pertukaran pada log.
 - d. Memperbarui tampilan GUI.
- Method `private void updateLabels_1005()`
 - Digunakan untuk memperbarui tampilan angka pada label GUI.
 - Method `public void resetHighlights_1005()`
 - Digunakan untuk mengembalikan warna label ke kondisi semula.
 - Method `private void reset_1005()`
 - Digunakan untuk menghapus seluruh input dan mengembalikan GUI ke kondisi awal.
 - Method `private Object arrayToString_1005`
 - Digunakan untuk mengubah array menjadi teks agar dapat ditampilkan pada area log.

- Method `public static void main (String[] args)`
 - Method utama digunakan untuk menjalankan GUI Bubble Sort menggunakan `SwingUtilities.invokeLater()` sehingga GUI dapat ditampilkan dengan baik.

2.4 Analisis Hasil

Berdasarkan program pada semua kelas yang dipraktikkan pada pekan ini, diperoleh beberapa hasil sebagai berikut:

1. Kelas `shellsort_2511531005`

Output :

```
sebelum :  
3 10 4 6 8 9 7 2 1 5  
sesudah shellsort :  
1 2 3 4 5 6 7 8 9 10
```

Hasil tersebut menunjukkan bahwa algoritma Shell Sort mampu mengurutkan data secara menaik dengan memanfaatkan konsep **gap (jarak)**. Pada awal proses, elemen dibandingkan menggunakan jarak tertentu sehingga perpindahan data dapat dilakukan lebih cepat dibandingkan Insertion Sort biasa. Setelah nilai gap semakin kecil hingga bernilai 1, proses pengurutan menjadi lebih optimal dan menghasilkan data yang sudah terurut. Keunggulan algoritma ini adalah mampu mempercepat proses sorting pada jumlah data yang lebih besar karena mengurangi jumlah pergeseran elemen.

2. Kelas `quicksort_2511531005`

Output :

```
data sebelum diurutkan :  
10 7 8 9 1 5  
data terurut quicksort :  
1 5 7 8 9 10
```

Berdasarkan hasil pengujian, Quick Sort mampu menghasilkan pengurutan yang cepat karena data dibagi menjadi sub-array kecil yang kemudian diurutkan secara rekursif. Algoritma ini memiliki performa yang sangat baik untuk data berukuran besar karena rata-rata kompleksitas waktunya mencapai $O(n \log n)$. Namun, Quick Sort juga memiliki kelemahan, yaitu performa dapat menurun apabila pemilihan pivot kurang optimal.

3. Kelas margesort_2511531005

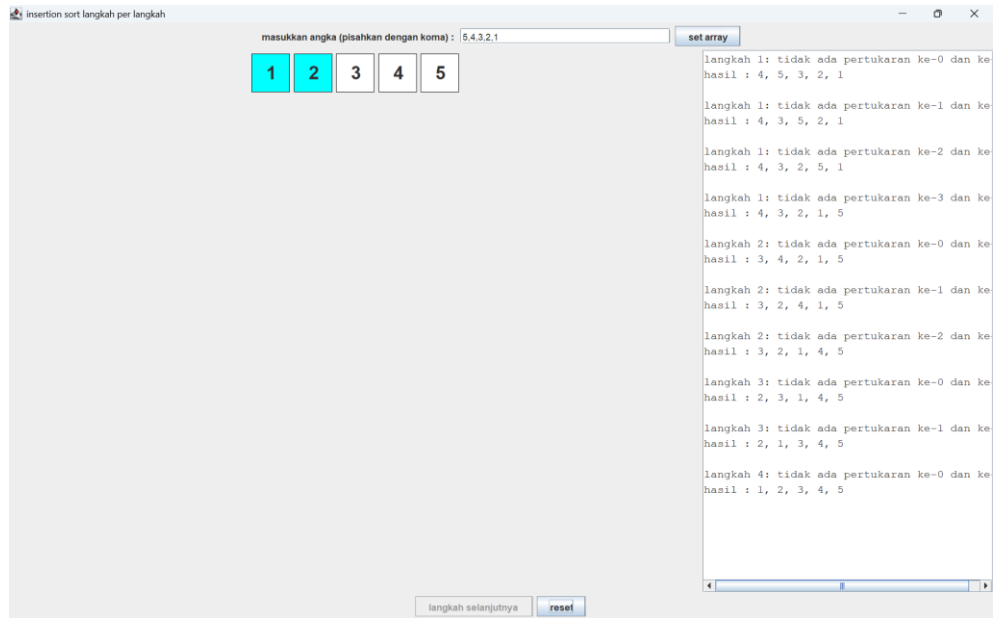
Output :

```
sebelum terurut :  
12 11 13 5 6 7  
sesudah terurut menggunakan margesort :  
5 6 7 11 12 13
```

Berdasarkan hasil praktikum, algoritma ini menunjukkan proses pengurutan yang stabil dan konsisten. Proses pembagian array dilakukan hingga tersisa satu elemen, kemudian dilakukan penggabungan kembali (merge) secara terurut. Kelebihan Merge Sort adalah performanya stabil dengan kompleksitas waktu $O(n \log n)$, sehingga efektif untuk data besar. Akan tetapi, algoritma ini membutuhkan memori tambahan karena menggunakan array sementara pada proses penggabungan.

4. Kelas GUI bubblesortGUI__2511531005

Output :



Berdasarkan output di atas, program GUI Bubble sort berhasil menampilkan visualisasi proses pertukaran elemen dan melihat perubahan warna saat elemen dibandingkan. Pada GUI ini, elemen yang dibandingkan akan diberi warna biru (cyan), sedangkan elemen yang ditukar akan berubah menjadi merah, sehingga pengguna dapat memahami proses Bubble Sort dengan lebih mudah.

BAB III

KESIMPULAN

3.1 Kesimpulan Praktikum

Berdasarkan praktikum yang telah dilakukan mengenai algoritma sorting pada pekan 8, dapat disimpulkan bahwa algoritma Shell Sort, Quick Sort, dan Merge Sort mampu mengurutkan data dengan metode yang berbeda namun menghasilkan output yang sama, yaitu data terurut secara menaik (ascending).

Shell Sort bekerja dengan memanfaatkan jarak (gap) tertentu dalam membandingkan data sehingga proses pengurutan menjadi lebih cepat dibandingkan metode insertion biasa. Quick Sort menggunakan konsep divide and conquer dengan pemilihan pivot untuk membagi data menjadi beberapa bagian kecil sebelum diurutkan kembali. Sedangkan Merge Sort bekerja dengan cara membagi array menjadi bagian kecil lalu menggabungkannya kembali dalam kondisi terurut.

Selain itu, implementasi Bubble Sort GUI memberikan pemahaman yang lebih baik mengenai proses sorting karena pengguna dapat melihat langkah-langkah pengurutan secara visual dan interaktif.

Dari hasil praktikum dapat dipahami bahwa setiap algoritma sorting memiliki kelebihan dan kekurangan masing-masing. Pemilihan algoritma yang tepat sangat bergantung pada jumlah data, kebutuhan efisiensi waktu, dan penggunaan memori dalam proses pengolahan data.

DAFTAR PUSTAKA

- [1] Oracle. (2025). *Java Platform, Standard Edition Documentation*. Oracle Corporation.
- [2] Goodrich, M. T., Tamassia, R., & Goldwasser, M. H. (2014). *Data Structures and Algorithms in Java* (6th ed.). Wiley.

TUGAS PRAKTIKUM PEKAN 8

Alasan memilih shellsort :

Karena shellsort logika dan prosesnya mudah dipahami.

A. Kode program

```
package pekan8_2511531005;
import java.util.Scanner;
class Lagu_2511531005{
    String judul_1005;
    String penyanyi_1005;
    int durasi_1005;

    Lagu_2511531005(String judul, String penyanyi, int durasi){
        this.judul_1005 = judul;
        this.penyanyi_1005 = penyanyi;
        this.durasi_1005 = durasi;
    }
}

public class Sorting_2511531005 {

    Lagu_2511531005[] dataLagu_1005 = new Lagu_2511531005[20];
    int jumlahLagu_1005 = 0;

    void inputData_1005() {
        dataLagu_1005[jumlahLagu_1005++] = new
Lagu_2511531005("kicau mania", "endy", 220);
        dataLagu_1005[jumlahLagu_1005++] = new Lagu_2511531005("mas
buahlil ganteng", "bang galid", 129);
        dataLagu_1005[jumlahLagu_1005++] = new Lagu_2511531005("dj
malam pagi", "karim", 123);
        dataLagu_1005[jumlahLagu_1005++] = new
Lagu_2511531005("bintang lima", "arip ganteng", 254);
        dataLagu_1005[jumlahLagu_1005++] = new Lagu_2511531005("so
asu", "nopal", 226);
        dataLagu_1005[jumlahLagu_1005++] = new
Lagu_2511531005("malu malu boy", "pahri", 283);
        dataLagu_1005[jumlahLagu_1005++] = new
Lagu_2511531005("tabola bale", "rehan", 190);
    }

    void tampilData_1005() {
        for (int i = 0; i < jumlahLagu_1005; i++) {
            System.out.println((i + 1)+". " +
dataLagu_1005[i].judul_1005 + " - "
+ dataLagu_1005[i].penyanyi_1005 + " - " +
dataLagu_1005[i].durasi_1005 + " detik");
        }
    }
}
```



```

        void shellSort_1005() {
            int n_1005 = jumlahLagu_1005;

            for (int gap_1005 = n_1005/2; gap_1005 > 0; gap_1005/=2) {
                for (int i = gap_1005; i < n_1005; i++) {
                    Lagu_2511531005 temp_1005 = dataLagu_1005[i];
                    int j;
                    for(j = i; j >= gap_1005 && dataLagu_1005[j -
gap_1005].judul_1005
                        .compareToIgnoreCase(temp_1005.judul_1005) > 0; j -= gap_1005) {
                            dataLagu_1005[j] = dataLagu_1005[j -
gap_1005];
                        }
                        dataLagu_1005[j] = temp_1005;
                    }
                }
            }

            public static void main(String[] args) {
                Scanner input_1005 = new Scanner (System.in);

                Sorting_2511531005 playlist_1005 = new
Sorting_2511531005();

                playlist_1005.inputData_1005();
                System.out.println("=== SHORTING PLAYLIST NIM : 2511531005
===");
                System.out.println("pilih algoritma (1=Shell, 2=Quick,
3=Merge) : ");
                int pilihan = input_1005.nextInt();

                System.out.println("\n data sebelum sorting :");
                playlist_1005.tampilData_1005();

                if (pilihan == 1) {
                    playlist_1005.shellSort_1005();

                    System.out.println("\n data setelah shellsort(judul
A-Z) : ");
                    playlist_1005.tampilData_1005();
                }
                else {
                    System.out.println("\n yang disuruh cuma satu metode
sorting\n pilih nomor 1");
                }
                input_1005.close();
            }
        }
    }
}

```

B. Penjelsan kode program

Program ini berfungsi untuk mengurutkan data playlist lagu menggunakan algoritma Shell Sort berdasarkan judul lagu secara alfabetis (A–Z). Program menggunakan array sebagai tempat penyimpanan data lagu dengan kapasitas maksimal 20 lagu. Setiap lagu memiliki atribut berupa judul lagu, penyanyi, dan durasi lagu.

1. Kelas `class Lagu_2511531005`

Kelas ini berfungsi untuk menyimpan data lagu seperti judul, penyanyi dan durasi lagu.

2. Array `Lagu_2511531005[] dataLagu_1005 = new Lagu_2511531005[20];`

Sintax untuk membuat array bernama dataLagu_1005 dengan kapasitas maksimum sebanyak 20.

3. Method `void inputData_1005()`

Method ini berfungsi untuk memasukkan data lagu ke dalam array.

4. Method `void tampilData_1005()`

Method ini berfungsi untuk menampilkan seluruh lagu di dalam array.

5. Method `void shellSort_1005()`

Ini merupakan program utama untuk mengurutkan lagu berdasarkan judul lagu dengan alfabetis (A-Z) dengan metode shellsort.

Berikut cara kerja shellsort :

- a. Program menentukan nilai gap awal, yaitu setengah dari jumlah data.
 - b. Setelah itu bandingkan data berdasarkan jarak tertentu (gap) bukan langsung berdekatan.
 - c. Jika urutan tidak sesuai alfabet, maka data akan dipindahkan.
 - d. Nilai gap diperkecil terus ($\text{gap} \neq 2$) sampai bernilai 1.
 - e. Saat $\text{gap} = 1$, proses akan menyerupai Insertion Sort, tetapi data sudah lebih rapi sehingga sorting menjadi lebih cepat.
6. Main method
- Method berfungsi sebagai pusat eksekusi program.

Langkah kerja pada method ini:

- a. Membuat objek Sorting_2511531005.
- b. Memanggil method inputData_1005().
- c. Menampilkan data sebelum sorting.
- d. Meminta pengguna memilih algoritma.
- e. Jika memilih angka 1, program menjalankan shellSort_1005().
- f. Menampilkan hasil setelah pengurutan.

C. Kompleksitas

Kompleksitas waktu (Time Complexity) dari algoritma Shell Sort bergantung pada urutan gap yang digunakan.

1. Best Case (Kasus Terbaik)

Ketika data hampir atau sudah terurut:

$[O(n \log n)]$

Pada kondisi ini, proses perpindahan data menjadi sedikit sehingga sorting berjalan lebih cepat.

2. Average Case (Kasus Rata-rata)

Pada kondisi data acak:

$[O(n^{1.5})]$

Shell Sort memiliki performa lebih baik dibandingkan insertion sort biasa karena membandingkan data dengan jarak tertentu.

3. Worst Case (Kasus Terburuk)

Ketika data benar-benar tidak terurut:

$[O(n^2)]$

Hal ini terjadi karena banyak proses perpindahan data selama pengurutan.

4. Space Complexity

Kompleksitas ruang dari Shell Sort adalah:

[O(1)]

Karena algoritma ini tidak membutuhkan array tambahan dan melakukan sorting langsung pada array asli (in-place sorting).

D. Output

```
<terminated> Sorting_2511531005 pada Application\ C:\Users\USER\
=== SHORTING PLAYLIST NIM : 2511531005 ===
pilih algoritma (1=Shell, 2=Quick, 3=Merge) :
1
|
data sebelum sorting :
1. kicau mania - endy - 220 detik
2. mas buahlil ganteng - bang galid - 129 detik
3. dj malam pagi - karim - 123 detik
4. bintang lima - arip ganteng - 254 detik
5. so asu - nopal - 226 detik
6. malu malu boy - pahri - 283 detik
7. tabola bale - rehan - 190 detik
```

```
data setelah shellsort(judul A-Z) :
1. bintang lima - arip ganteng - 254 detik
2. dj malam pagi - karim - 123 detik
3. kicau mania - endy - 220 detik
4. malu malu boy - pahri - 283 detik
5. mas buahlil ganteng - bang galid - 129 detik
6. so asu - nopal - 226 detik
7. tabola bale - rehan - 190 detik
```

```
=== SHORTING PLAYLIST NIM : 2511531005 ===
pilih algoritma (1=Shell, 2=Quick, 3=Merge) :
2
|
data sebelum sorting :
1. kicau mania - endy - 220 detik
2. mas buahlil ganteng - bang galid - 129 detik
3. dj malam pagi - karim - 123 detik
4. bintang lima - arip ganteng - 254 detik
5. so asu - nopal - 226 detik
6. malu malu boy - pahri - 283 detik
7. tabola bale - rehan - 190 detik

yang disuruh cuma catu metode sorting
pilih nomor 1
```